

Chương 15: Xử lý Chuỗi bằng RNN và CNN

Giảng viên: Tiến sĩ Lê Hoàng Vũ
Đại Học Đông Á

Ngày 10 tháng 8 năm 2026

Gemini Notebook



1. Dữ liệu Chuỗi và Mạng nơ-ron hồi quy (RNN)



2. Ứng dụng và Kiến Trúc Dựa Trên Chuỗi



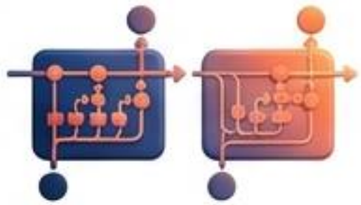
3. Huấn luyện RNN: BPTT



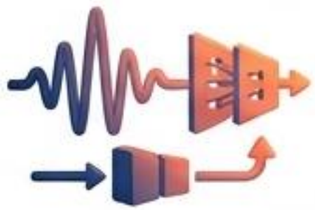
4. Dự báo Chuỗi Thời Gian (Time Series)



5. Mạng RNN Sâu (Deep RNNs)



6. Kiến trúc Tế bào Nhớ: LSTM và GRU



7. WaveNet (Mạng CNN 1D cho Chuỗi)



8. Tổng kết

Gemini Notebook

Quá khứ (Dữ liệu truyền thống): Dữ liệu độc lập và phân phối chuẩn (i.i.d).

Hiện tại (Dữ liệu chuỗi): Tính thứ tự và sự phụ thuộc thời gian là cốt lõi.

Ví dụ điển hình:



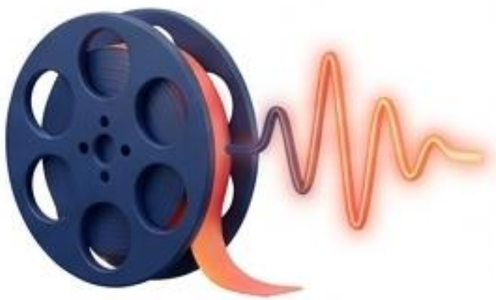
Chuỗi thời gian kinh tế
(Giá cổ phiếu qua các ngày, GDP hằng năm)



Dữ liệu y tế
(Điện tâm đồ - ECG, điện não đồ - EEG)



Ngôn ngữ tự nhiên - NLP
(Một câu là một chuỗi các từ có cấu trúc ngữ pháp)



Âm thanh / Video
(Chuỗi các khung hình và sóng âm theo thời gian)

Gemini Notebook

Giới hạn của Mạng Truyền thẳng (Feedforward Networks)

Mạng truyền thẳng (như MLP, CNN truyền thống) có 3 điểm yếu lớn:



Kích thước đầu vào bị cố định (Fixed-size input).



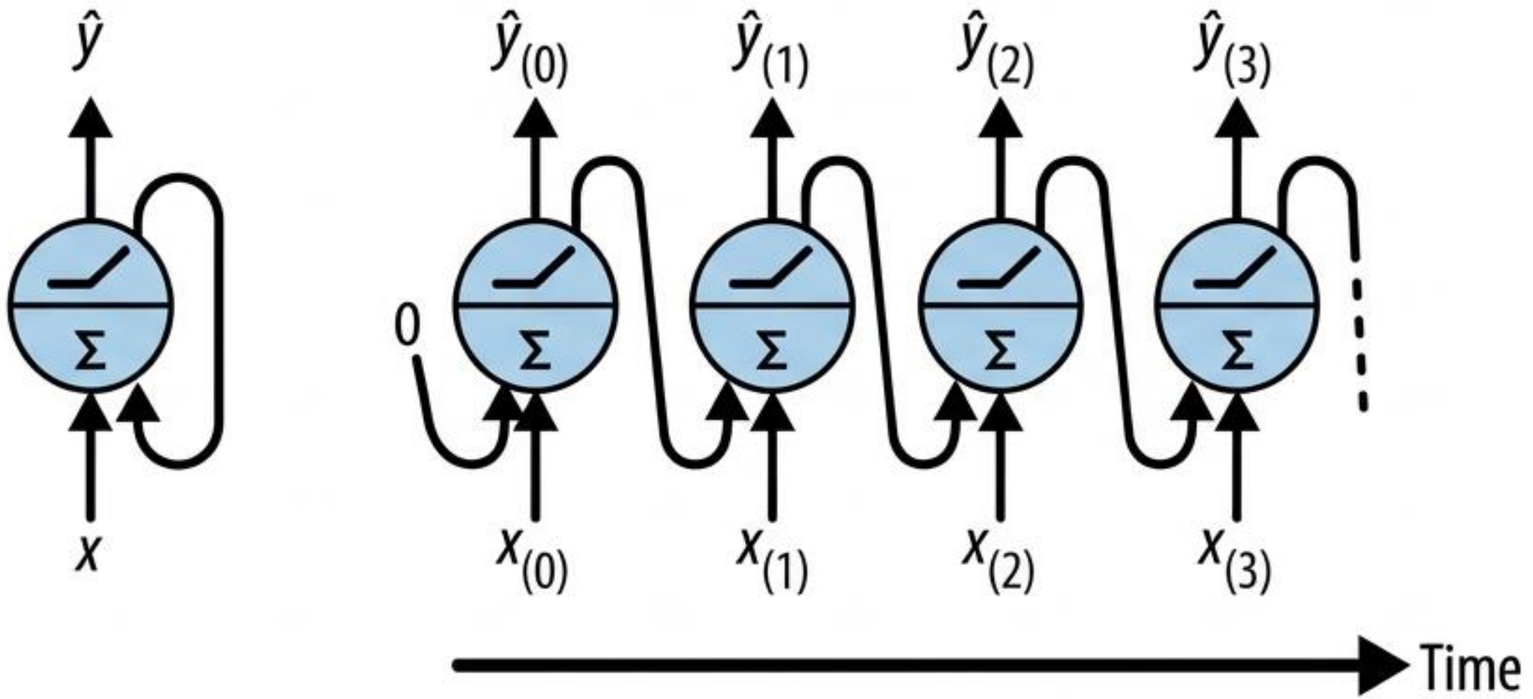
Không có bộ nhớ (Memoryless): Mỗi lần nhận đầu vào mới, mạng quên hoàn toàn các đầu vào trước đó.



Không nắm bắt được phụ thuộc xa (Long-term dependencies) trừ khi nạp một lượng lớn đầu vào cùng lúc.

Gemini Notebook

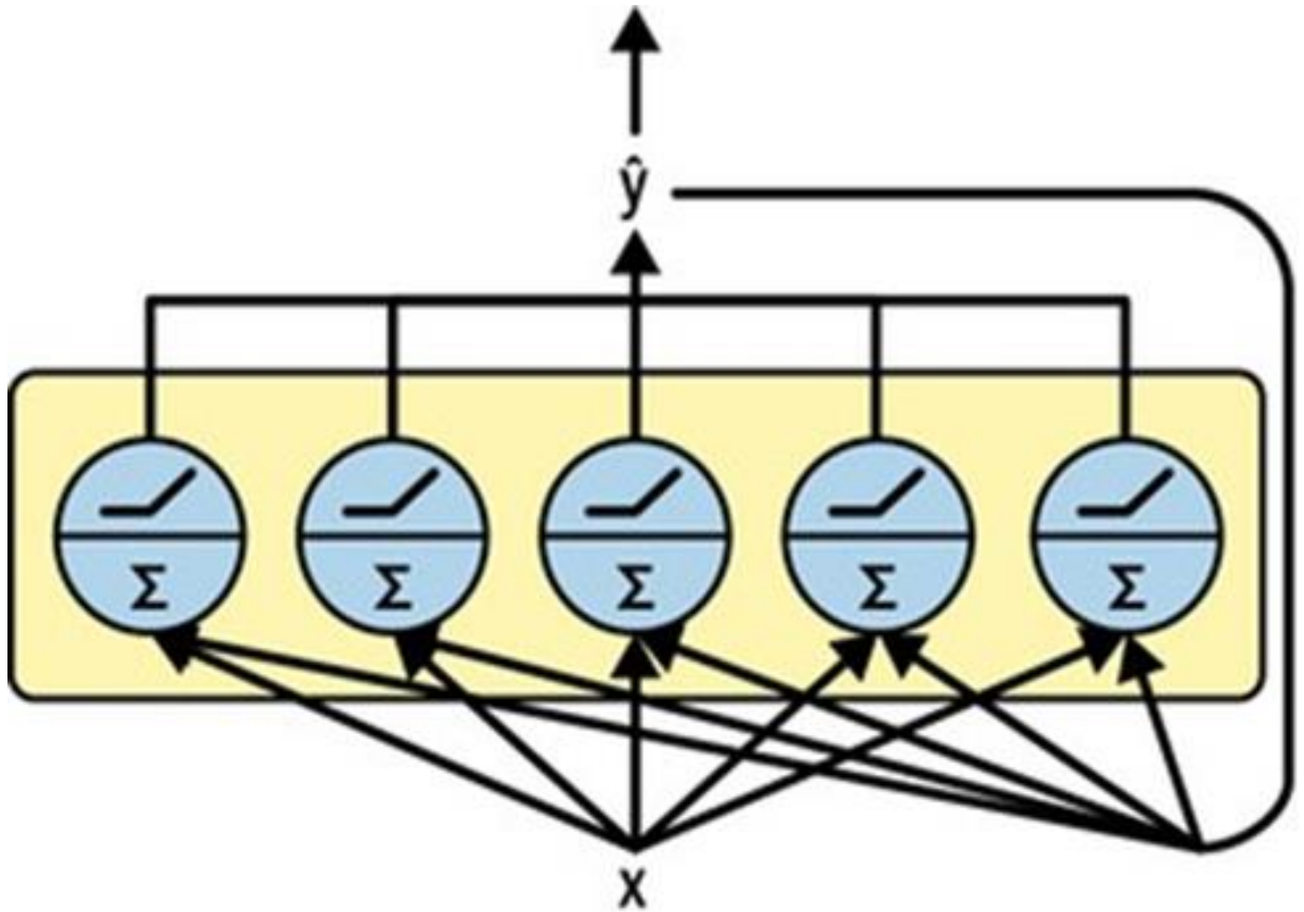
Giải pháp: Mạng Nơ-ron Hồi quy (RNN)

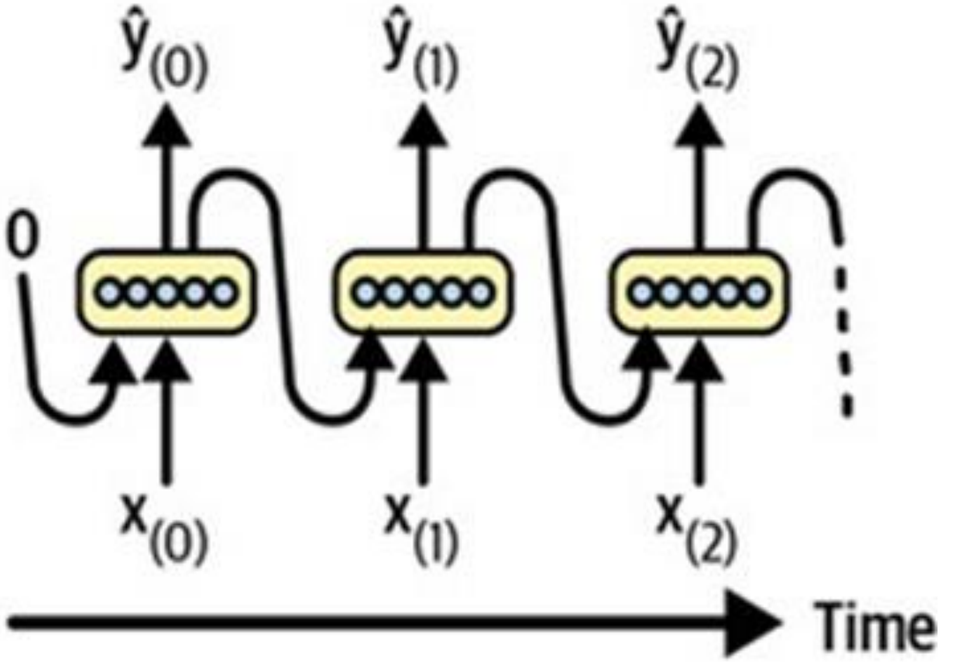


Hình 15-1: RNN cơ bản và phiên bản mở rộng

- **Giải pháp**: Cần một mạng có khả năng lưu giữ trạng thái của những gì nó đã thấy.
- Một nơ-ron hồi quy đơn giản không chỉ nhận đầu vào từ dữ liệu x(t) tại bước t, mà còn nhận trạng thái ẩn h(t-1) từ bước trước đó.
- Mạng có thể xử lý chuỗi độ dài tùy ý vì nó thực hiện một phép tính lặp lại (recurrent) ở mỗi bước.

Gemini Notebook


ŷ
Σ
Σ
Σ
Σ
Σ
x


ŷ(0)
ŷ(1)
ŷ(2)
0
x(0)
x(1)
x(2)
Time

Trạng thái Ẩn (Hidden State) và Tính Toán Toán Học

$$\mathbf{h}(t) = \phi(W_{hx}\mathbf{x}(t) + W_{hh}\mathbf{h}(t-1) + \mathbf{b})$$

Đầu vào tại bước t.

Trạng thái ẩn từ bước t-1 (mang thông tin "trí nhớ").

Hàm kích hoạt (thường dùng tanh thay vì ReLU ở RNN chuẩn để tránh phát nổ giá trị).

Trọng số cho đầu vào.

Trọng số cho trạng thái ẩn.

Vector độ lệch (bias).

Gemini Notebook

Cơ chế Chia sẻ Trọng số (Weight Sharing)



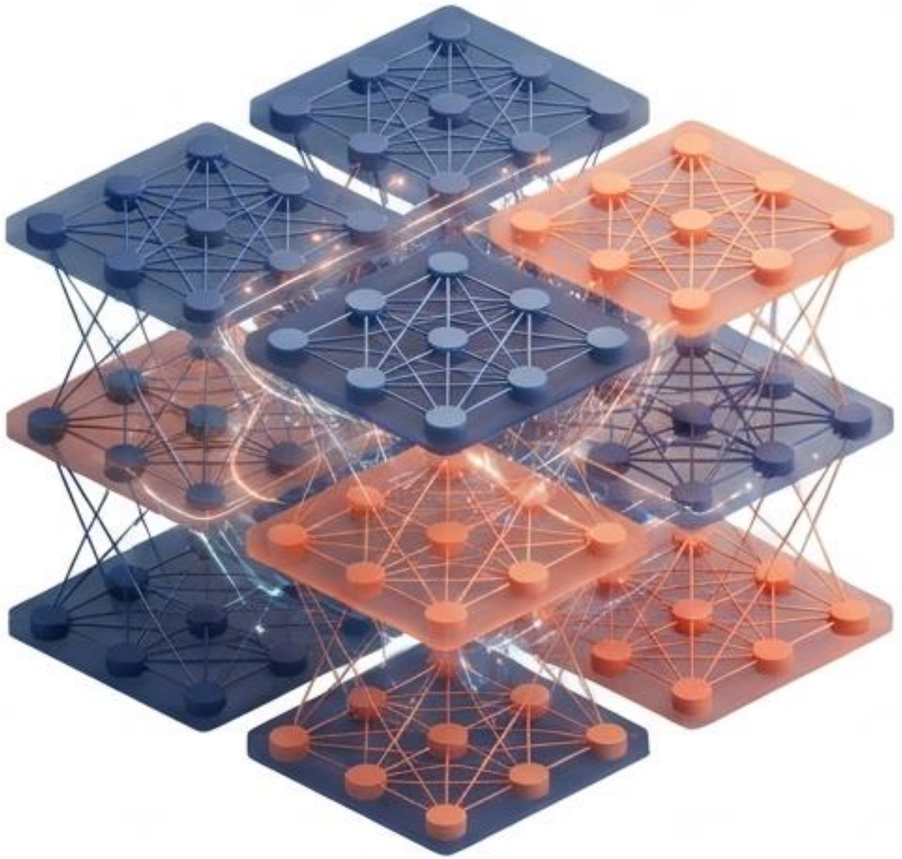
Khác với mạng MLP nơi mỗi lớp có bộ trọng số riêng, RNN sử dụng cùng một bộ ma trận trọng số W_{hx}, W_{hh} cho tất cả các bước thời gian t.



- Lợi ích:
- Giảm số lượng tham số đáng kể.
- Tránh quá khớp (overfitting).
- Có khả năng tổng quát hóa cho các chuỗi có độ dài chưa từng thấy trong tập huấn luyện.

Gemini Notebook

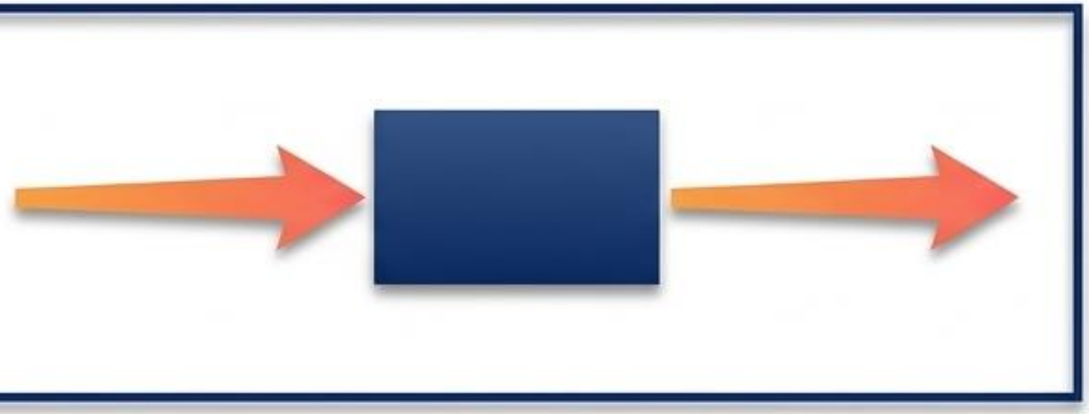
Từ Nơ-ron đơn đến Lớp Nơ-ron (RNN Layer)



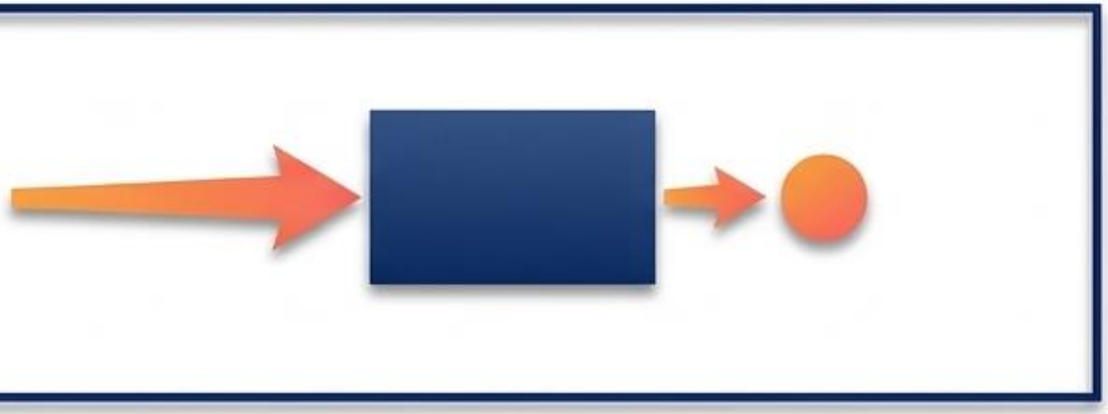
- **Mô hình thực tế:** Hình 15-2 biểu diễn một lớp nơ-ron RNN thay vì chỉ một nơ-ron đơn.
- Thay vì nhận một đặc trưng, mạng nhận một **vector đặc trưng x(t)**.
- Lớp này trả về **vector đầu ra y(t)** (thường chính là **h(t)**).
- **Mini-batch:** Ký hiệu vector giúp mở rộng sang **phép tính ma trận** để xử lý nhiều chuỗi cùng lúc.

Gemini Notebook

Phân loại Bài toán xử lý Chuỗi



Sequence-to-Sequence (Đồng bộ): Đầu vào N bước, xuất N bước. (VD: Dự báo thời tiết liên tục).



Sequence-to-Vector: Đầu vào N bước, chỉ xuất 1 kết quả ở cuối chuỗi. (VD: Phân loại văn bản, Sentiment analysis).



Vector-to-Sequence: Đầu vào 1 giá trị, sinh ra chuỗi độ dài N. (VD: Tạo chú thích ảnh - Image Captioning).

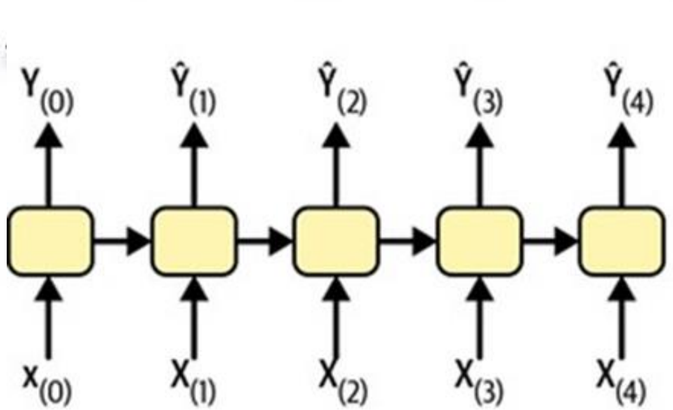


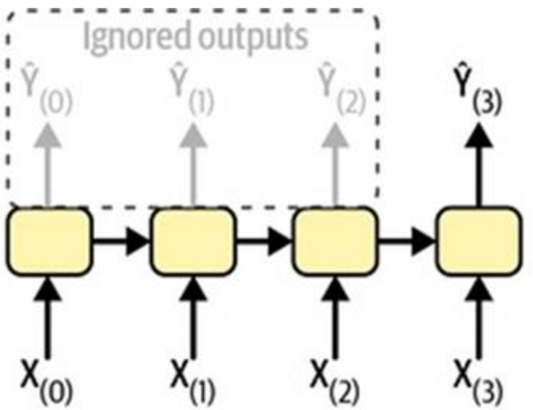
Encoder-Decoder (Delayed Seq-to-Seq): Nhận chuỗi N, dịch qua chuỗi M. (VD: Dịch máy, Trả lời câu hỏi).

Gemini Notebook

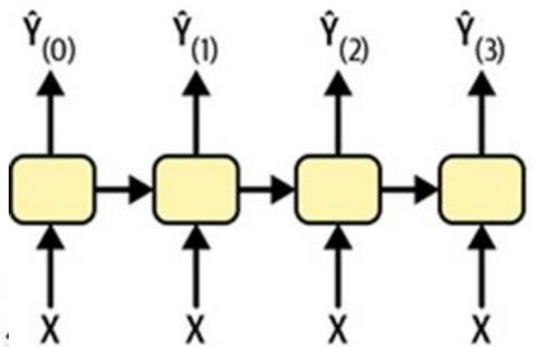
Minh Họa 4 Dạng Kiến Trúc RNN Cốt Lõi

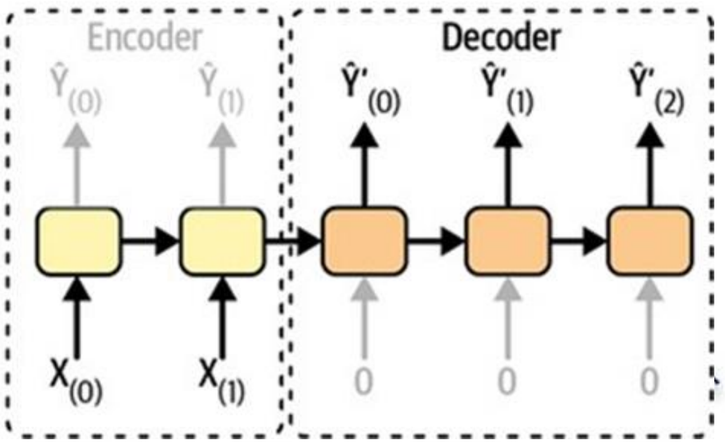
Seq-to-Seq (đồng bộ).





Seq-to-Vec. Đầu ra chỉ quan tâm ở bước cuối.





Hình 15-4: Các dạng Seq2Seq, Seq2Vec, Vec2Seq

Vec-to-Seq. Đầu vào duy nhất ở t=0.

Mã hóa - Giải mã. Chờ hết câu rồi mới dịch.

Gemini Notebook

Ứng dụng: Dự báo Chuỗi Thời Gian (Time Series)



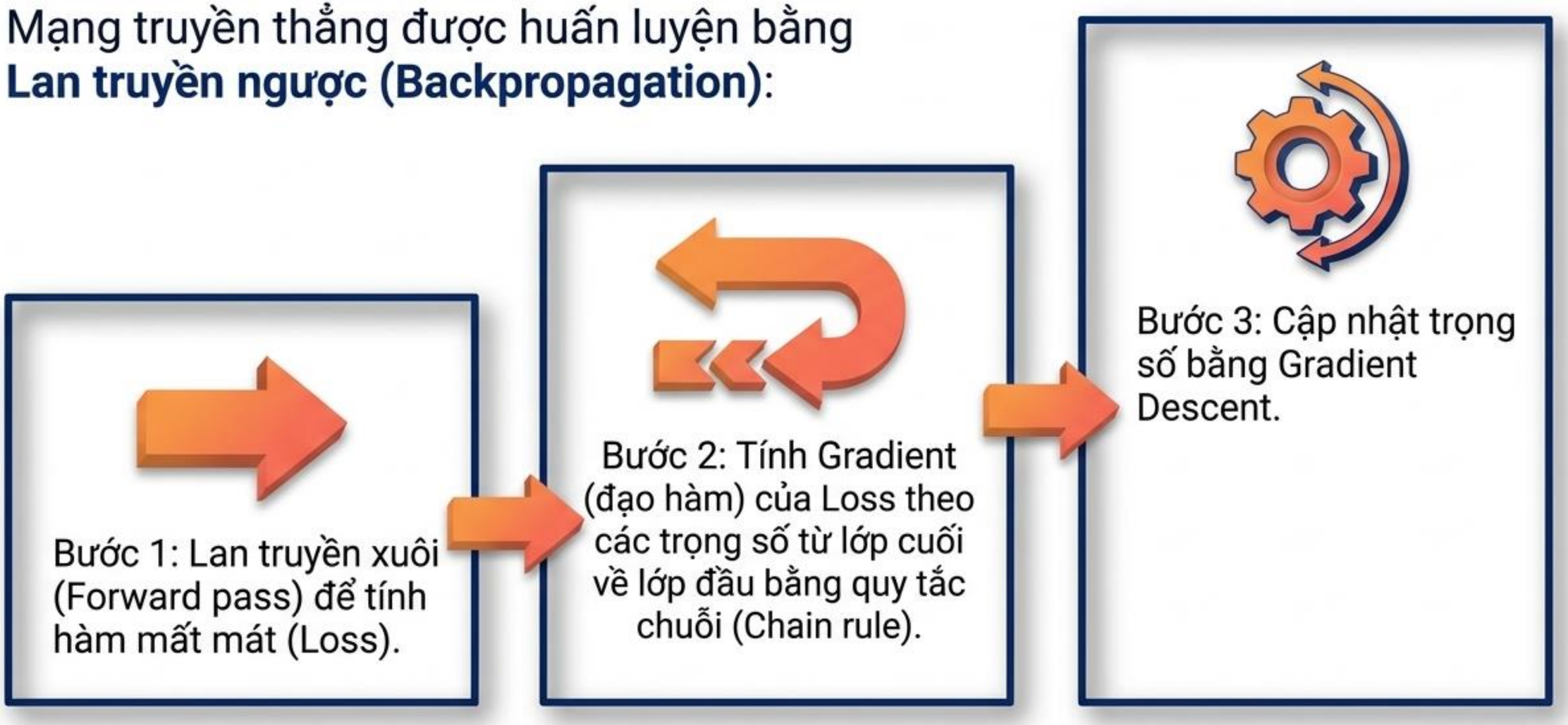
Kiến trúc dùng nhiều nhất cho Time Series Forecasting (minh họa từ Hình 15-3: Seq-to-Seq trễ trong dự báo).

- Kỳ vọng mạng dự đoán tương lai y(0), y(1)... dựa trên quá khứ x(0), x(1)...
- Đầu ra y(t) tại thời điểm t là kết quả dự báo cho x(t+1).
- Cơ chế: Do đó mạng bị trễ 1 bước (shifted by 1 step).

Gemini Notebook

Nền tảng: Huấn luyện Mạng Truyền thẳng

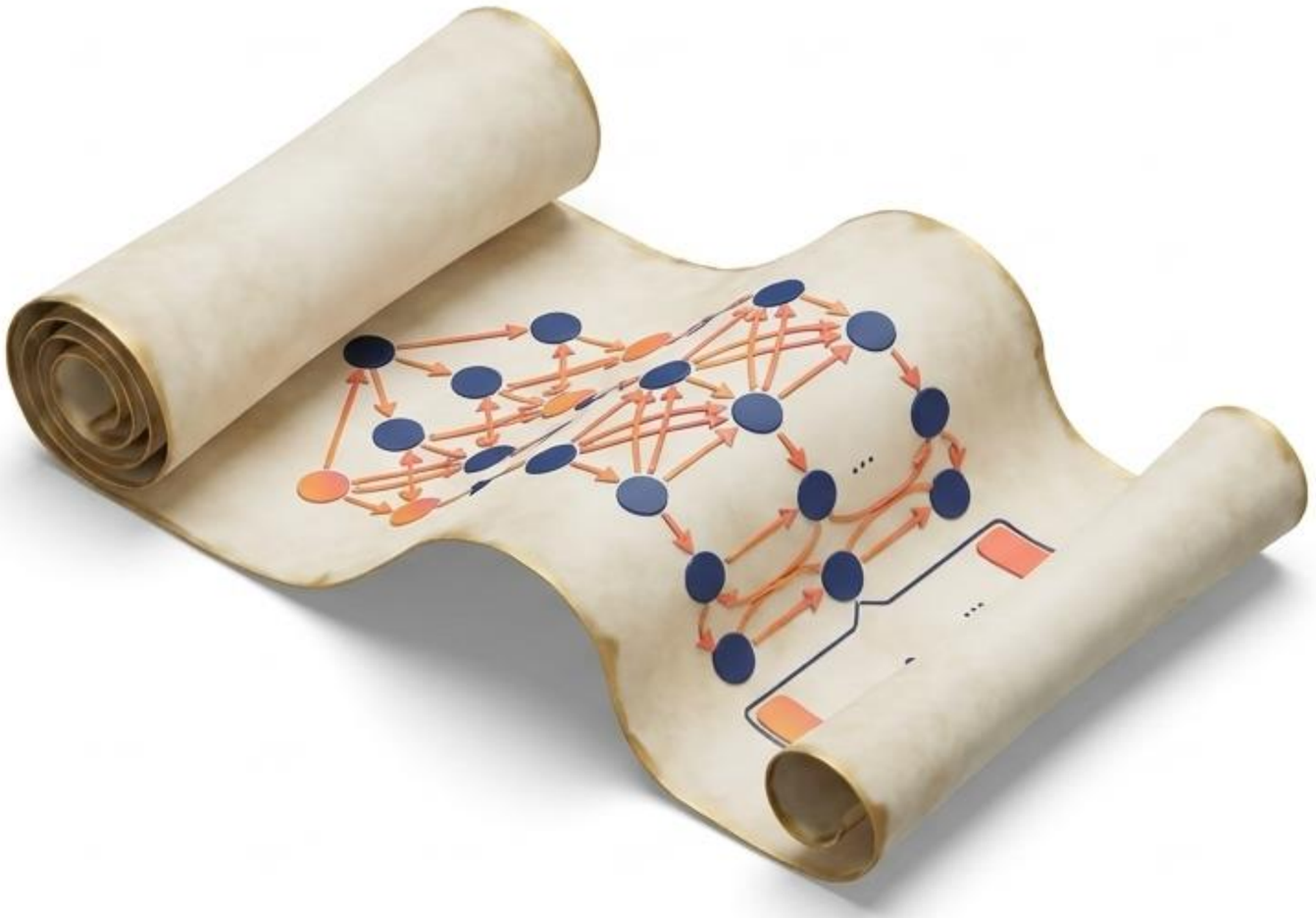
Mạng truyền thẳng được huấn luyện bằng **Lan truyền ngược (Backpropagation)**:



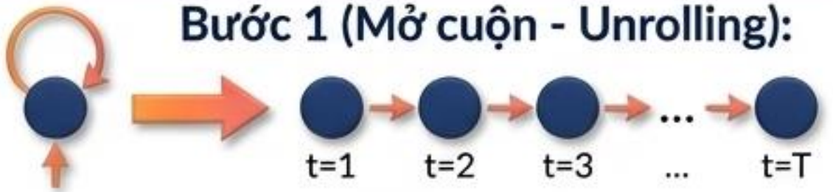
Gemini Notebook

BPTT - Lan truyền ngược qua thời gian

Chuỗi thời gian t = 1...T làm cấu trúc mạng giống như một mạng rất sâu T lớp -> Cần kỹ thuật BPTT.



Bước 1 (Mở cuộn - Unrolling):



Tạm thời xem RNN như một mạng Feedforward khổng lồ với T lớp.

Bước 2 (Lan truyền xuôi):



Tính Loss trên mọi bước thời gian (hoặc chỉ bước cuối).

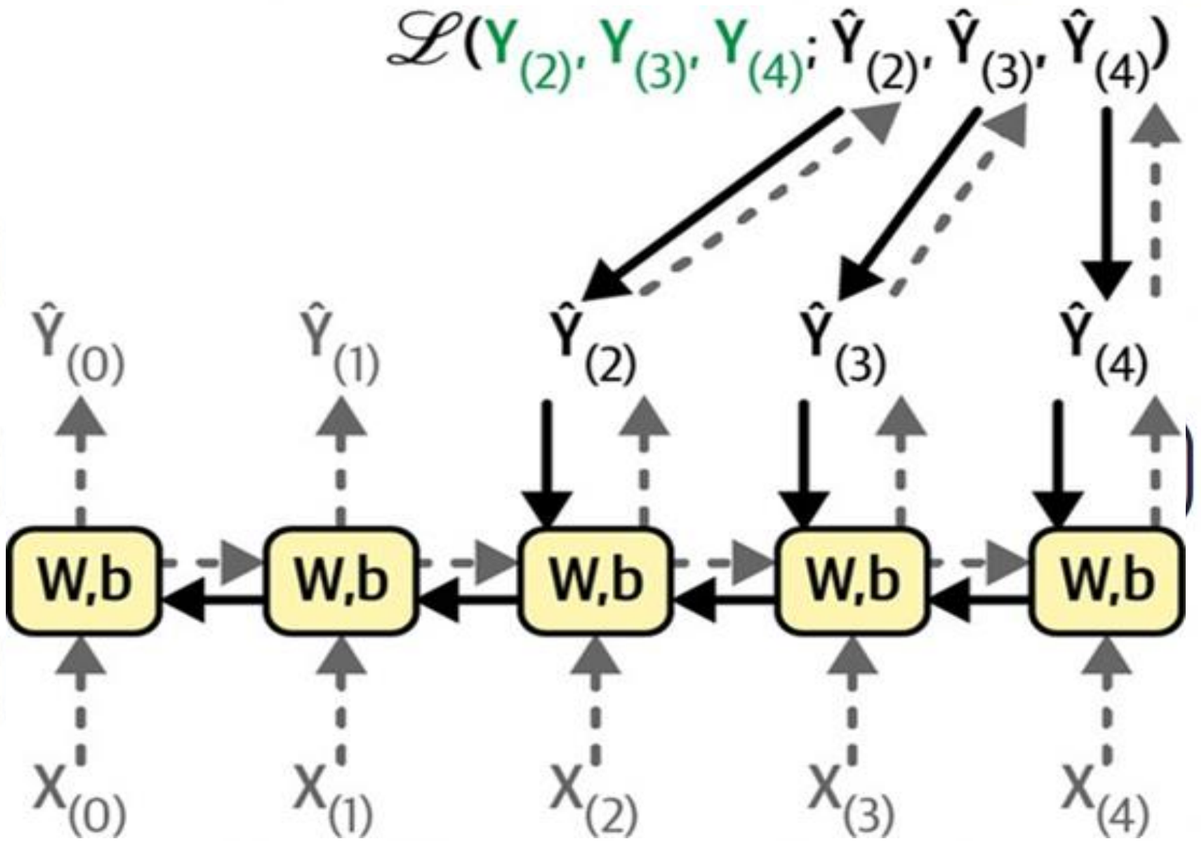
Bước 3 (Lan truyền ngược):



Gradient lan truyền ngược dọc theo chiều thời gian từ t = T về t = 0.

Gemini Notebook

Cơ chế Hoạt động của BPTT



Hình 15-5: BPTT - Lan truyền ngược qua thời gian

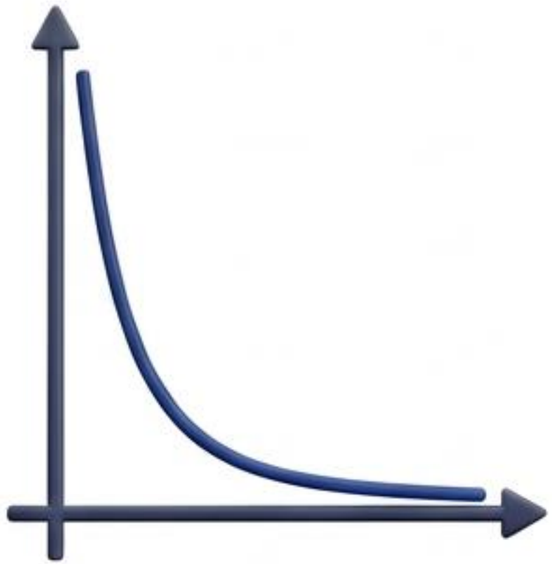
Luồng Gradient: Mũi tên liền nét dày mô tả luồng Gradient ngược từ hàm mất mát L.

Tổng cập nhật: Do ma trận trọng số W được chia sẻ ở mọi t, gradient cuối cùng để cập nhật W là tổng $\nabla W(t)$.

Cảnh báo hiệu năng: Phép tính này cực kỳ đắt đỏ và tốn RAM nếu độ dài chuỗi rất lớn.

Gemini Notebook

Giới hạn cốt lõi: Vấn đề Triệt tiêu Gradient



Nguyên nhân: Trong BPTT, đạo hàm phải được nhân với ma trận trọng số W_{hh} liên tục T lần (qua T bước thời gian).

Phương trình: $\nabla \boldsymbol{h}^{(0)} \propto (\boldsymbol{W}_{\text{hh}})^{T} \nabla \boldsymbol{h}^{(T)}$

Nếu giá trị riêng lớn nhất của $W_{\text{hh}} < 1$, gradient tiến dần về 0 theo hàm số mũ.



Hậu quả: Mạng không thể học được phụ thuộc xa (Ví dụ: Từ ở đầu câu văn quá dài không có ảnh hưởng gì tới dự đoán ở cuối câu). Trí nhớ mạng bị "ngắn hạn".

Gemini Notebook

Hiện tượng Bùng nổ Gradient (Exploding Gradients)



Nguyên nhân

Nếu giá trị riêng lớn nhất của Whh > 1, gradient bị nhân lên theo cấp số nhân.



Hậu quả

Phát nổ thành NaN hoặc vô cực.

Cập nhật trọng số quá lớn, làm mất ổn định hoàn toàn mạng, Loss dao động điên cuồng.



Giải pháp

Gradient Clipping

Cắt cụt gradient nếu nó vượt quá một ngưỡng nhất định trước khi cập nhật. Thường xuyên được áp dụng mặc định trong các Framework như Keras/PyTorch.

Gemini Notebook

Đặc trưng của Dữ liệu Chuỗi thời gian (Time Series)



Nhiễu trắng (White noise):

Các biến động ngẫu nhiên.



Tính thời vụ (Seasonality):

Các mẫu lặp lại đều đặn (Ngày, tuần, tháng).



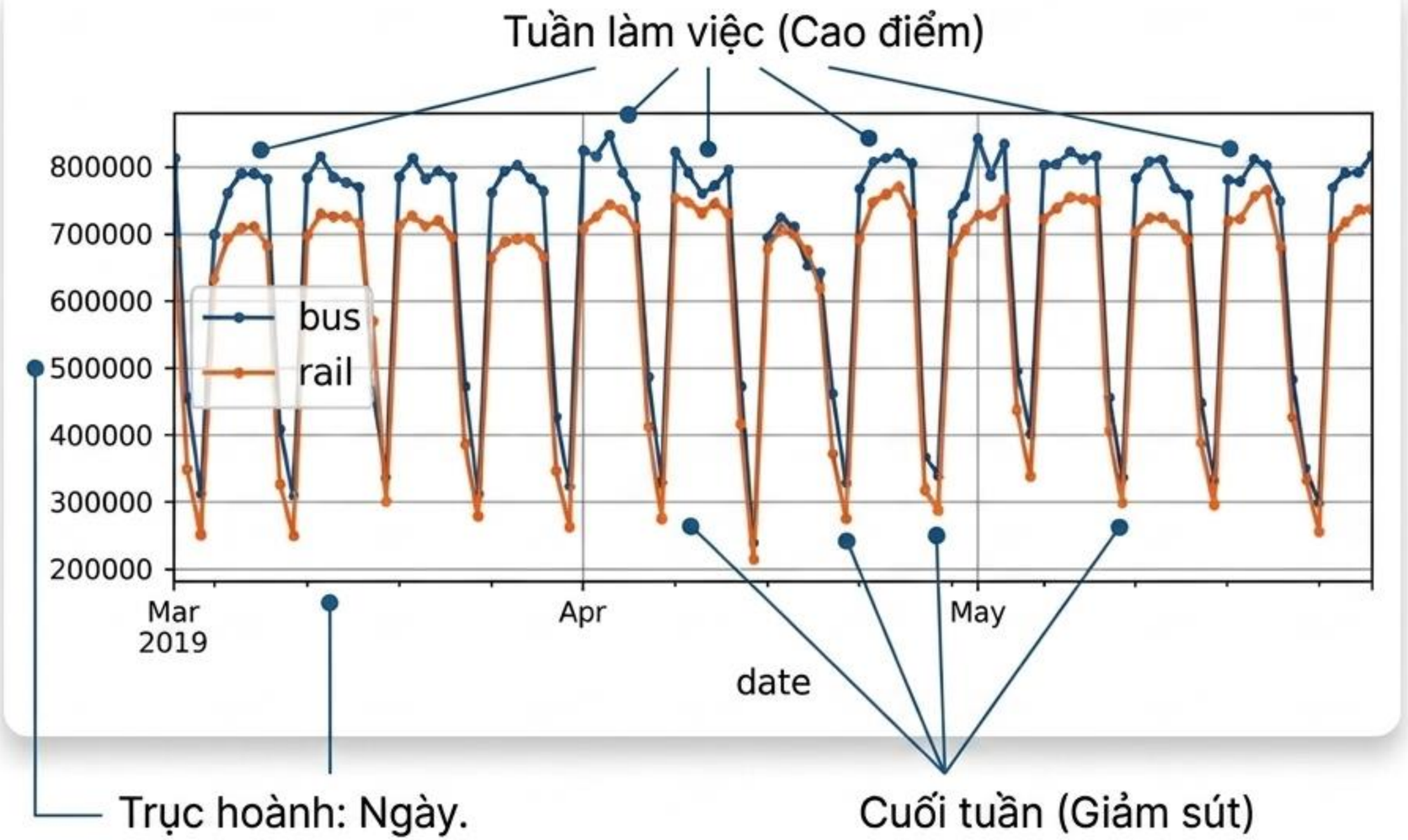
Xu hướng (Trend):

Tăng trưởng hoặc suy giảm chậm trong thời gian dài.

Tiền xử lý dữ liệu để loại bỏ những yếu tố này đôi khi hiệu quả hơn để mạng nơ-ron tự học từ đầu.

Gemini Notebook

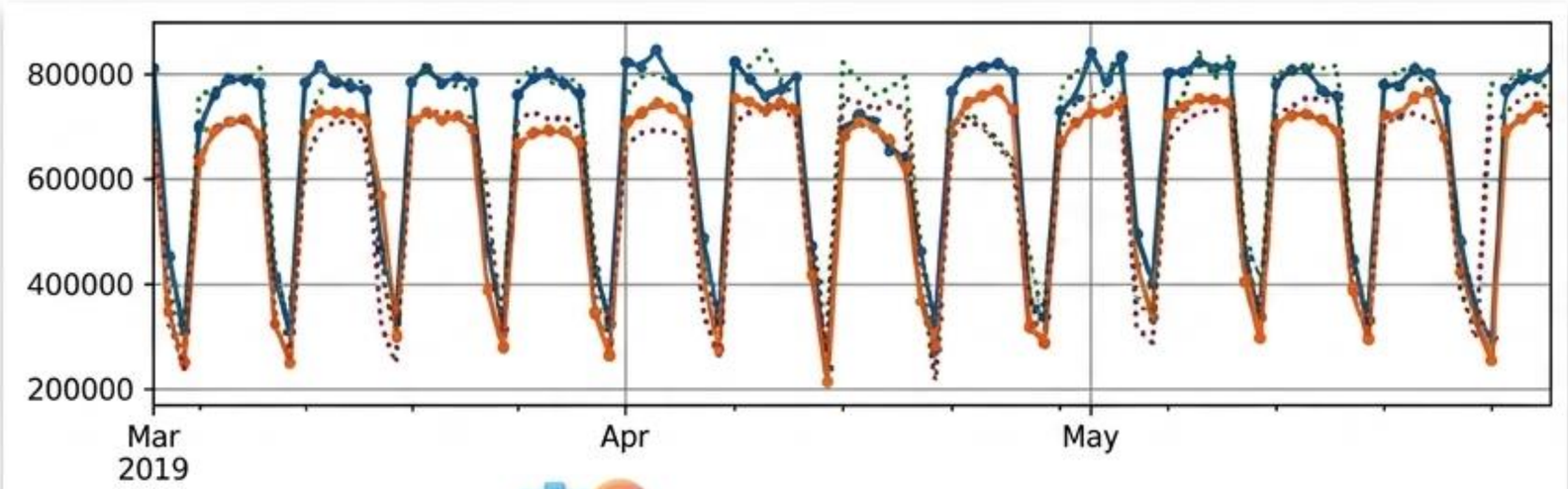
Phân tích Dữ liệu Thực tế: Hành khách Chicago (2019)



Biểu đồ cho thấy các **biến động liên tục và nhịp nhàng.** Mỗi **'khối' lởm chởm** tương ứng với **một tuần làm việc và cuối tuần.**

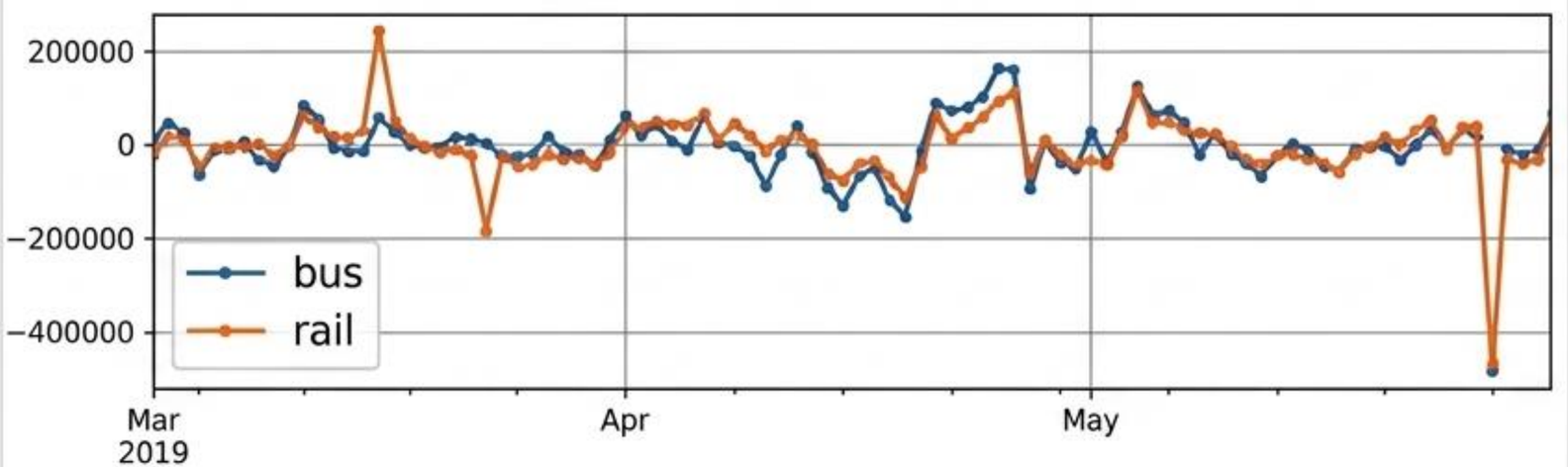
Gemini Notebook

Kỹ thuật Tiền xử lý: Khử tính Thời vụ Hàng tuần



Biểu đồ trên: Chuỗi gốc so với chuỗi bị dịch (trễ 7 ngày).

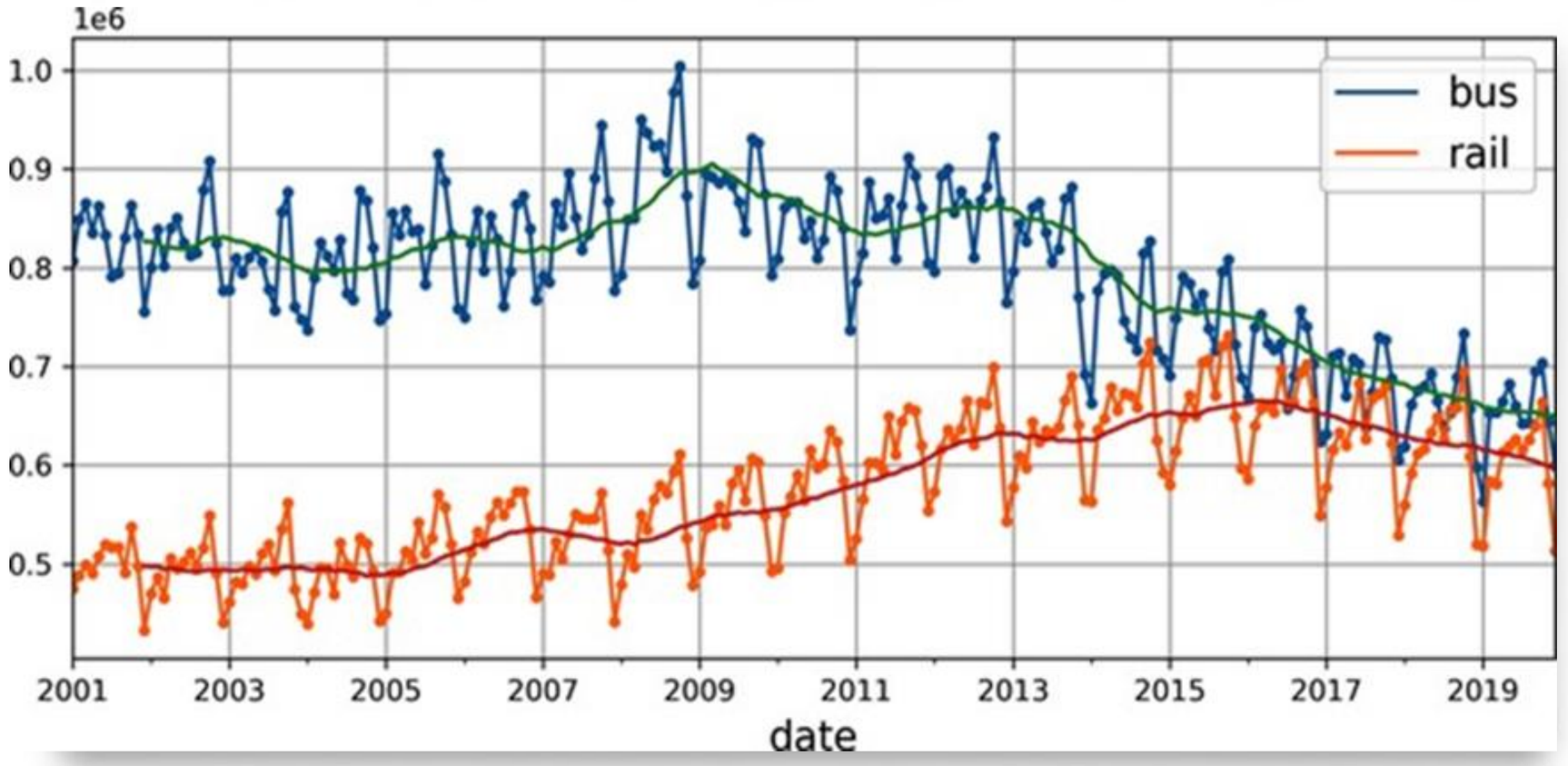
$y_t - y_{t-7}$ (Phép tính sai phân trễ 7 ngày).



Biểu đồ dưới: Chuỗi sai phân mượt mà, stationary (dừng). Đã loại bỏ được nhiễu do khác biệt giữa ngày thường và cuối tuần.

Gemini Notebook

Biến động Vĩ mô & Nhu cầu 'Trí nhớ Dài hạn'



Mở rộng ra 15 năm (2001-2015), biểu đồ cho thấy rõ ràng các ngày lễ hội khiến lượng khách sụt giảm lớn hàng năm, và tổng thể hành khách biến động vĩ mô (Macro Trend).



Để RNN mô hình hóa được điều này, nó cần 'Trí nhớ Dài Hạn' vươn xa hàng trăm bước — điều này là cực kỳ khó nếu chỉ dùng SimpleRNN.

Gemini Notebook

Kỹ thuật Tiền xử lý: Sai phân 12 tháng (Dài hạn)



Áp dụng sai phân 12 tháng liên tục. (Hình 15-9).

Chuỗi kết quả cân bằng hơn quanh mốc 0.

Mạng RNN hiện đại có thể **tự phát hiện** các trend này. Tuy nhiên, trong thực tế, đưa vào chuỗi sai phân thường giúp mô hình **hội tụ nhanh hơn** và **dự đoán chính xác hơn** so với việc dùng dữ liệu gốc hỗn độn.

Gemini Notebook

Khởi tạo SimpleRNN với Keras

```python
import tensorflow as tf
from tensorflow import keras

# Tao mang RNN 1 lop, 1 hidden unit
model = keras.Sequential([
    keras.layers.SimpleRNN(1, input_shape=[None, 1])
])

model.compile(loss="mse", optimizer="adam")
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
```

API Huấn luyện mô phỏng nơ-ron hồi quy cơ bản.

Gemini Notebook

Giải mã Tham số Keras (Keras Parameters Explained)



```
SimpleRNN(1)
```

Tham số 1 là số lượng nơ-ron trạng thái ẩn (**dimensionality of output space**). Chọn 1 vì đây là dự đoán 1 bước tiếp theo (đại lượng vô hướng).



```
input_shape=[None, 1]
```

- **None**: Chiều dài chuỗi (Time steps). Keras cho phép chuỗi dài ngắn tùy ý.
- **1**: Số chiều (Features) tại mỗi bước thời gian (Chỉ có 1 đặc trưng: lượng hành khách).



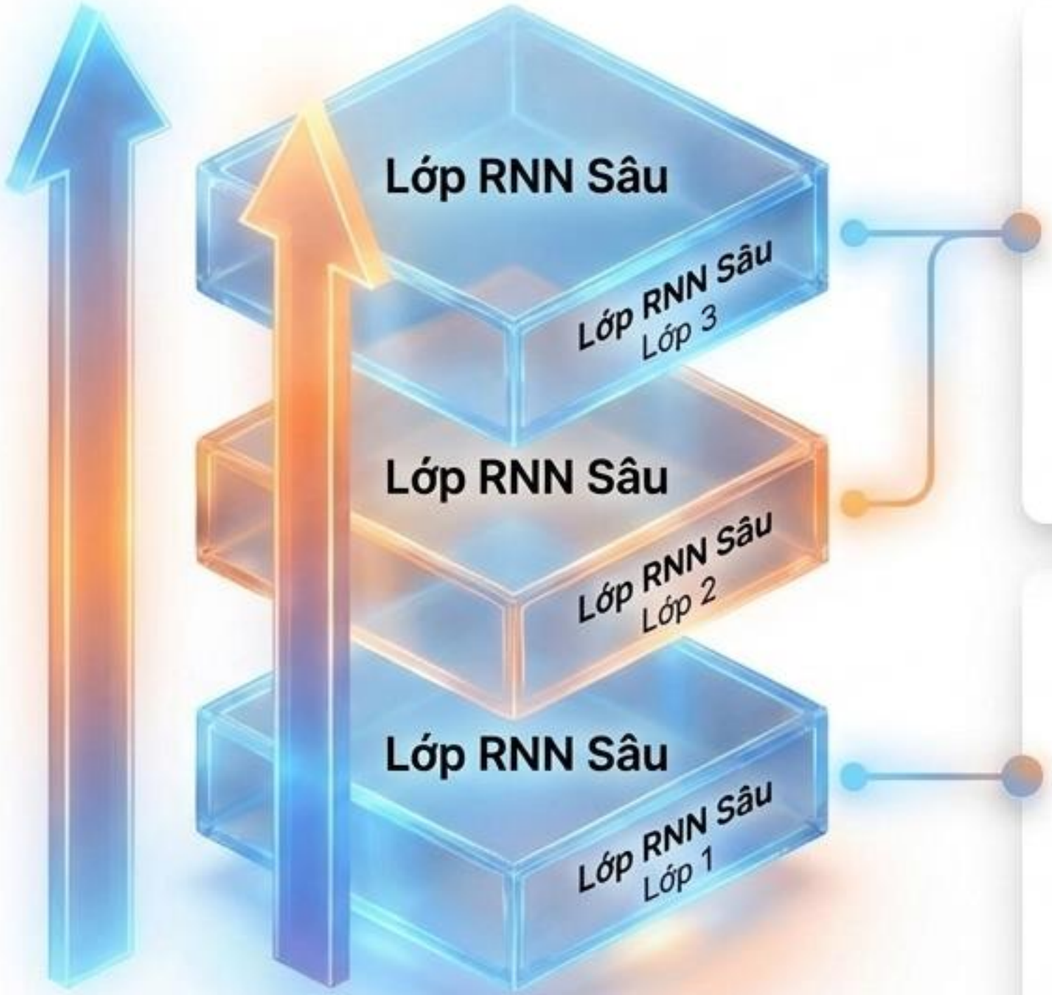
Hàm Loss

Thường dùng cho dự báo chuỗi là **mse** (Mean Squared Error) hoặc **mae** (Mean Absolute Error).

Gemini Notebook

Kiến trúc Deep RNN (Mạng RNN Sâu)

Tương tự MLP Sâu, ta xếp chồng (stacking) các lớp RNN lên nhau tạo thành Deep RNN.



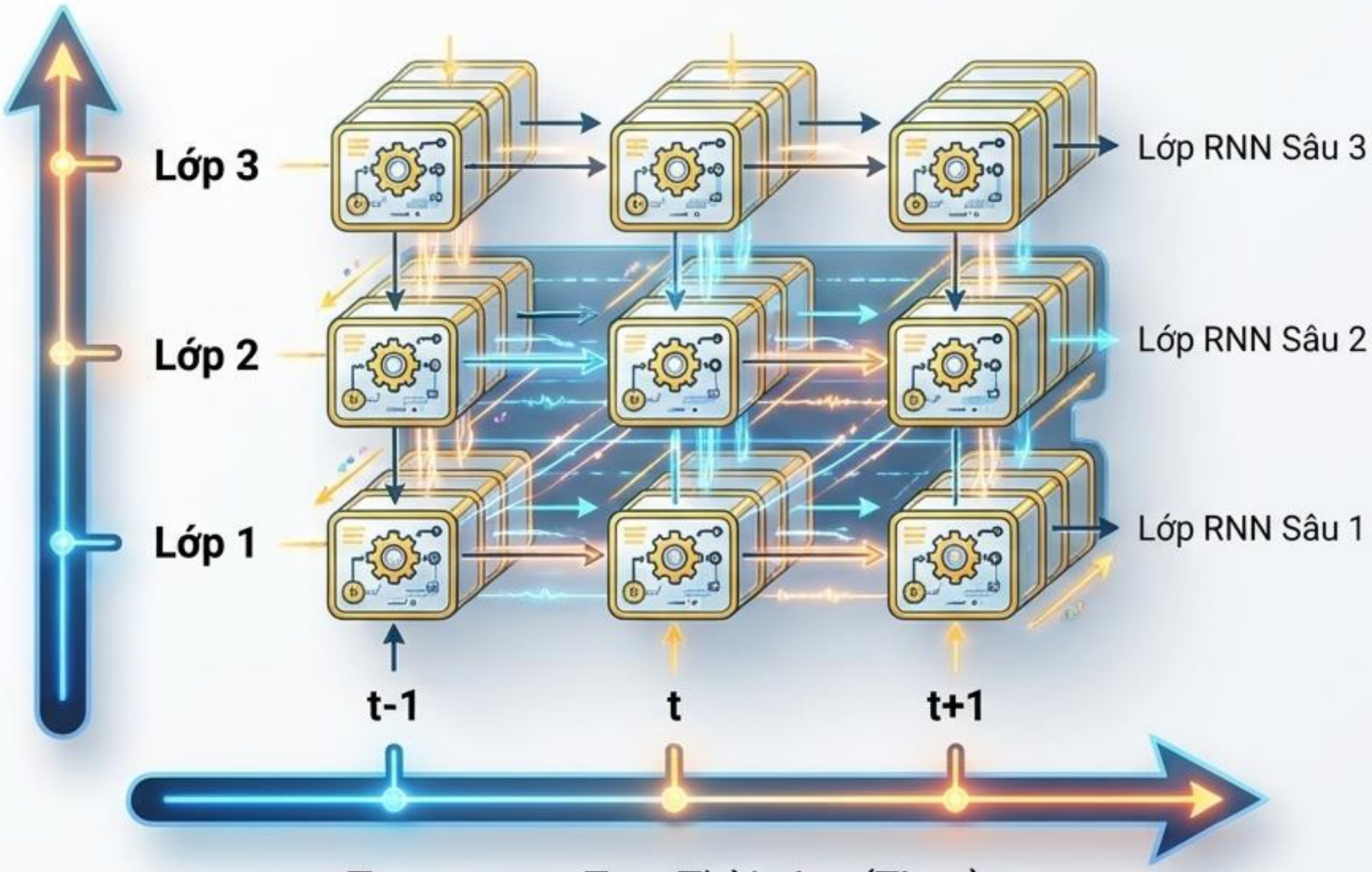
2. Đầu ra trạng thái ẩn **h(t)** của lớp dưới làm đầu vào cho lớp trên tại cùng thời điểm t.

1. Lớp dưới cùng nhận dữ liệu đầu vào.

Gemini Notebook

Sơ đồ Lan truyền trong Deep RNN

Trục dọc:
Trục Không gian / Chiều sâu (Space/Depth)



Trục ngang: Trục Thời gian (Time)

Gemini Notebook

Keras Code: Xây dựng Deep RNN



YÊU CẦU QUAN TRỌNG: Để lớp RNN dưới có thể đẩy chuỗi output lên lớp RNN trên, nó **BẮT BUỘC** phải bật **return_sequences=True**.

```
deep_model = keras.Sequential([
    # Lớp RNN 1: Trả về chuỗi cho lớp 2
    keras.layers.SimpleRNN(20, return_sequences=True,
        input_shape=[None, 1]),
    # Lớp RNN 2: Trả về chuỗi cho lớp cuối
    keras.layers.SimpleRNN(20, return_sequences=True),

    # Lớp RNN cuối: Chỉ trả về vector cuối cũng (Seq2Vec)
    keras.layers.SimpleRNN(1)
])
```

Gemini Notebook

Chiến lược Dự báo Đa bước (Multi-step Forecasting)

Dự đoán tuần tự (Autoregressive)

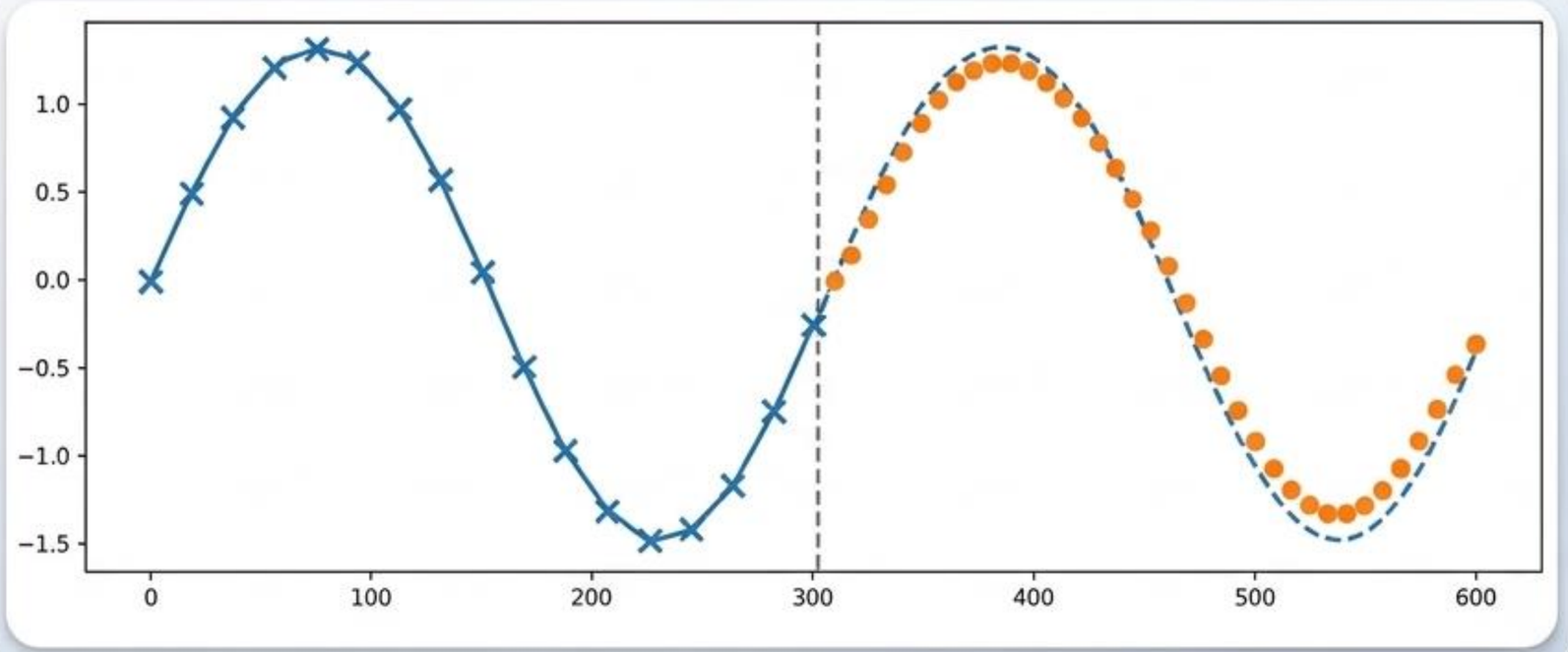
Dự đoán t+1, gộp vào mảng đầu vào để đoán t+2, lặp lại 10 lần.

(Nhược điểm: Dễ tích lũy sai số).

Dự đoán đồng thời

Đổi lớp cuối thành Dense(10) xuất 10 giá trị cùng lúc.

(**Ưu điểm**: Thường ổn định hơn).



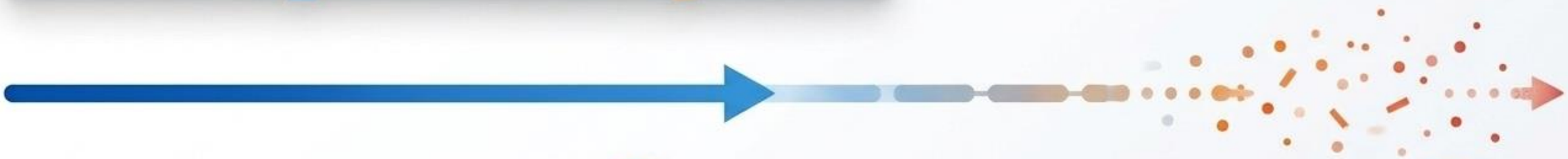
- Dấu × (Đầu vào)
- Đường đứt nét (Target)
- Dấu chấm tròn (Dự đoán của mạng)

Mạng bám sát rất tốt đường đi tự nhiên của sóng dốc.

Gemini Notebook

Hạn chế của SimpleRNN: Phụ thuộc xa (Long-Term Dependencies)

"Tôi sống ở Pháp từ nhỏ, sau 20 năm chuyển đến Mỹ, tôi vẫn nói trôi chảy tiếng [X]."



The Problem:

Mạng cần nhớ chữ “Pháp” ở quá khứ **rất xa** để đoán chữ “Pháp” ở hiện tại.

Vanishing Gradient:

Trọng số W nếu nhỏ thì Vanishing Gradient làm chữ “Pháp” bốc hơi sau **vài bước thời gian**. SimpleRNN rất tồi trong việc nắm bắt phụ thuộc xa.

Gemini Notebook

Sự ra đời của Tế bào LSTM (Long Short-Term Memory)

Phát minh bởi Hochreiter và Schmidhuber (1997) để tạo ra **"Trí nhớ Dài hạn"**.



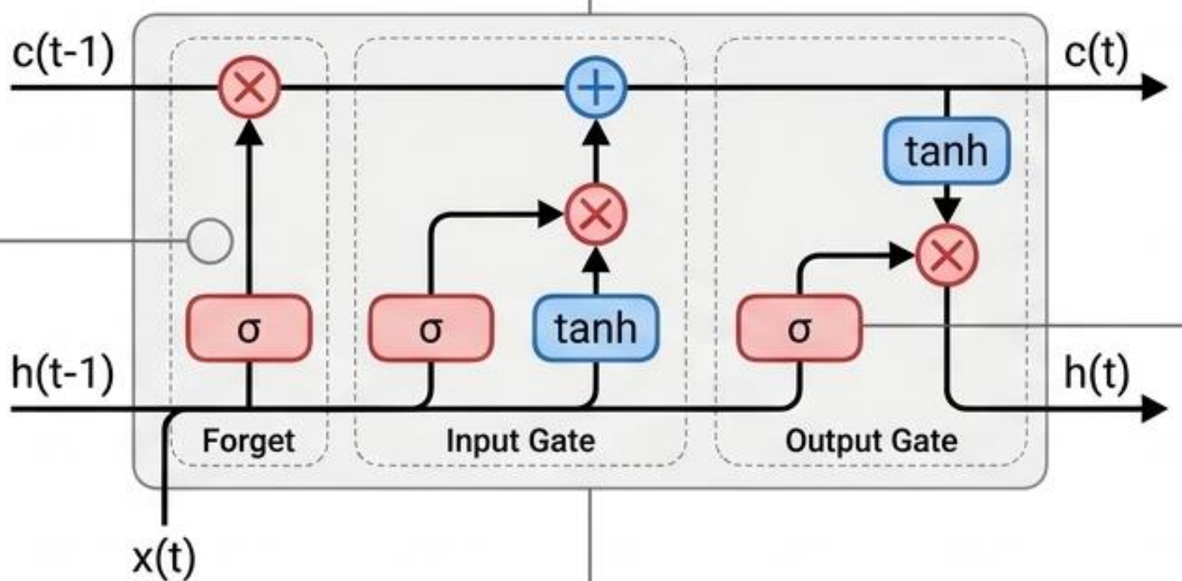
Thay vì chỉ truyền một Trạng thái ẩn h(t), LSTM **duy trì thêm một đường truyền phụ** gọi là **Trạng thái tế bào** (Cell State - c(t)).

c(t) giống như một đường ray cao tốc xuyên suốt, cho phép thông tin truyền đi rất xa mà không suy giảm. Các **Cổng (Gates)** kiểm soát việc ghi, đọc, xóa dữ liệu trên đường ray này.

Gemini Notebook

Giải phẫu Cấu trúc Tế bào LSTM

Đường ray ngang trên cùng:
Trạng thái tế bào c(t).



Legend

⊗, ⊕ là phép nhân phần tử và phép cộng vector.

Gate Valve Callout

Cổng (Gate) sử dụng hàm kích hoạt Sigmoid (σ) xuất giá trị [0, 1].

Hoạt động như một cái "van": 0 là đóng hoàn toàn, 1 là mở hoàn toàn.

Đường ray dưới:
Trạng thái ẩn h(t).

Gemini Notebook

Phân tích 3 Cổng của LSTM (Phần 1)



1. Cổng Quên (Forget Gate - f(t))

- ✓ Nhận vào **h**(t-1) và **x**(t), tính toán bằng hàm Sigmoid.
- ✓ Quyết định phần nào của Trạng thái dài hạn **c**(t-1) sẽ bị 'bỏ đi'.

Ví dụ: Thấy chủ ngữ mới thì quên chủ ngữ cũ.



2. Cổng Cập nhật & Ghi (Input Gate - i(t), g(t))

g(t): Tạo ra vector giá trị ứng viên mới (dùng tanh).

i(t): Quyết định (bằng hàm Sigmoid) mức độ quan trọng của các ứng viên này để cộng vào **c**(t) làm thông tin mới.

Gemini Notebook

Phân tích 3 Cổng của LSTM (Phần 2)

Cập nhật Trạng thái Tế bào (c(t))

$$c(t) = f(t) \otimes c(t-1) + i(t) \otimes g(t)$$

(Quên đi quá khứ không cần thiết + Nhớ thêm thông tin mới)



3. Cổng Xuất (Output Gate - o(t))

Quyết định phần nào của Trạng thái Tế bào **c(t)** sẽ được đọc ra và đưa vào Trạng thái ẩn **h(t)**.

$$o(t) = \sigma(W_o[x(t), h(t-1)])$$

$$h(t) = o(t) \otimes \tanh(c(t))$$

Gemini Notebook

Sử dụng LSTM trong Keras

Cực kỳ đơn giản, chỉ cần thay SimpleRNN bằng LSTM. Mọi công thức cổng phức tạp bên trên đã được Keras tối ưu bằng thư viện CUDNN dưới nền.

```
model = keras.Sequential([
    keras.layers.LSTM(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.LSTM(20),
    keras.layers.Dense(1)
])
```



Hiệu năng: LSTM giải quyết tốt chuỗi dài khoả hàng trăm bước thời gian, vượt xa SimpleRNN (chỉ hiệu quả với < 20 bước).

Gemini Notebook

Tế bào GRU (Gated Recurrent Unit)

GRU (2014) là một nỗ lực đơn giản hóa cấu trúc của LSTM.

Difference 1



1. Gộp Trạng thái:

GRU không có đường ray c(t) riêng biệt. Nó gộp chung Trạng thái ẩn h(t) và Trạng thái tế bào làm một.

Difference 2



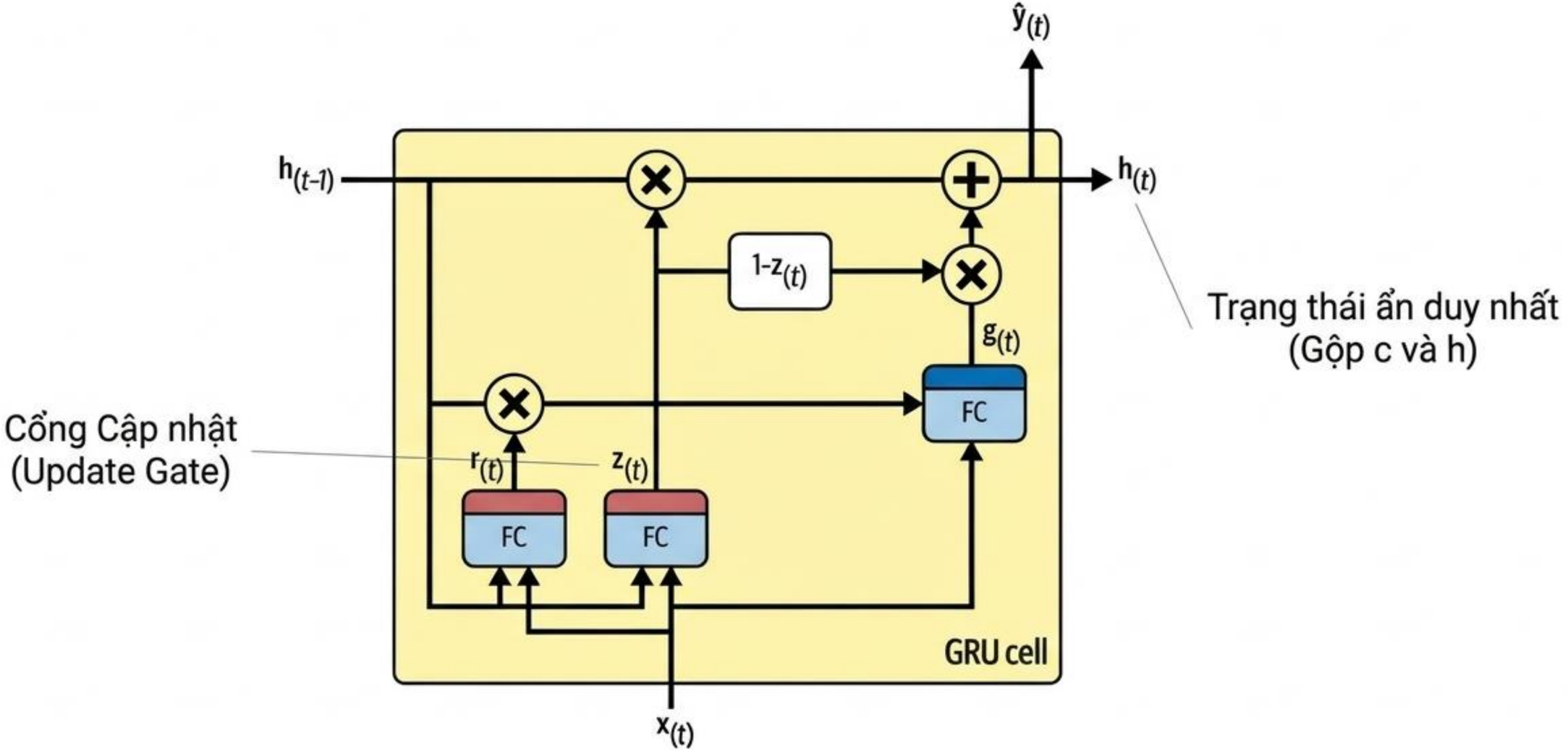
2. Gộp Cổng:

Gộp Cổng Quên và Cổng Cập nhật thành một Cổng Cập nhật (Update Gate - z(t)).

- Nếu z = 1: Lưu mới
- Nếu z = 0: Giữ cũ

Gemini Notebook

Cấu tạo Tế bào GRU



Hình 15-13: Cấu tạo Tế bào GRU

Gemini Notebook

So sánh LSTM và GRU

Đặc điểm	LSTM	GRU
Năm ra đời	1997	2014
Trạng thái duy trì	h_t và c_t	Chỉ h_t
Số lượng Cổng	3 (Quên, Nhập, Xuất)	2 (Cập nhật, Reset)
Tốc độ huấn luyện	Chậm 	Nhanh hơn 
Số lượng tham số	Lớn (Nặng RAM)	Thấp hơn (~ 25% ít hơn)
Độ chính xác	Tốt với chuỗi rất phức tạp	Ngang ngửa LSTM ở đa số Data



Lời khuyên thực tiễn: Bắt đầu thử nghiệm với GRU vì nó nhanh. Nếu thấy hiệu năng có vẻ bị thắt nút, chuyển sang LSTM xem có cải thiện không.

Gemini Notebook

Từ Tuần tự sang Song song



Điểm yếu chết người của RNN (kể cả LSTM/GRU)

Tính toán tuần tự.

Bạn không thể tính bước thời gian t = 100 nếu chưa tính xong bước t = 99.

Hậu quả: Không thể chạy song song trên phần cứng GPU một cách hiệu quả.



Giải pháp: Mạng Tích chập (CNN 1D)

Kernel trượt độc lập với nhau -> Hoàn toàn song song hóa được.

Cách thức: Dùng CNN 1D (chỉ quét theo chiều dài thời gian) để trích xuất đặc trưng chuỗi.

Gemini Notebook

Vượt qua giới hạn bằng Tích chập Giãn nở

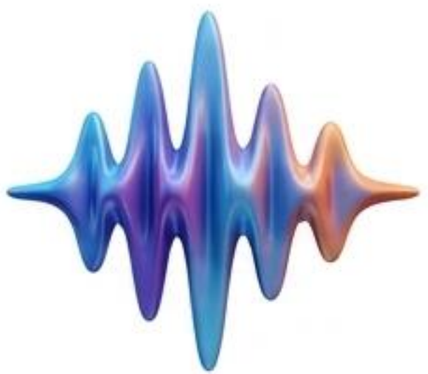


Vấn đề của CNN 1D thông thường

- Bộ lọc (Kernel) có kích thước **nhỏ** (VD: size 3) -> Mỗi nơ-ron lớp 1 chỉ nhìn thấy 3 bước thời gian.
- Để nhìn được 100 bước thời gian (Trường thụ cảm - Receptive field lớn) -> Phải **xếp chồng hàng chục lớp** CNN 1D + Pooling.
- Kết quả: Rất nặng nề, mất thông tin chuỗi chi tiết. Cần cơ chế quét khác!



Giải pháp: WaveNet của DeepMind

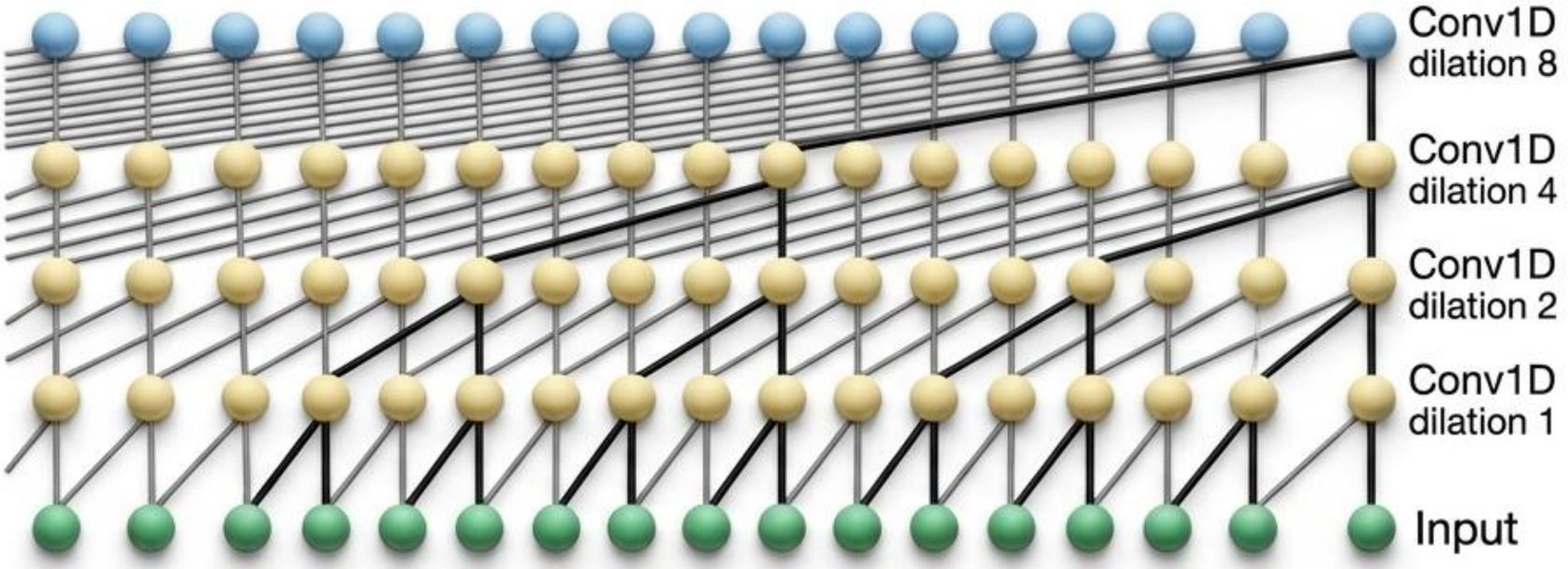


Chuyên tạo **giọng nói Audio** (chuỗi cực kỳ dài, tới 16,000 mẫu/giây).

Thay vì CNN thường, nó sử dụng **Tích chập giãn nở** (**Dilated Convolutions**).

Gemini Notebook

Tích chập Giãn nở (Dilated Convolutions) trong WaveNet



Hình 15-14: *Kiến trúc WaveNet (Dilated Conv)*

Tỷ lệ giãn (Dilation rate) tăng dần theo lũy thừa 2^n (1, 2, 4, 8...):

- **Lớp 1 (rate=1):** Kernel trượt từng ô liền kề (Giống hệt Conv1D thường).
- **Lớp 2 (rate=2):** Kernel bỏ qua 1 ô xen kẽ -> 1 nơ-ron bao phủ được khoảng cách 4 ô.
- **Lớp 3 (rate=4):** Kernel bỏ qua 3 ô xen kẽ -> 1 nơ-ron nhìn được 8 ô.

Nhờ tăng rate theo lũy thừa, mạng chỉ cần **10 lớp** là đã **có Trường thụ cảm** bao phủ được **1024 bước thời gian** mà số lượng **tham số vẫn cực kỳ nhỏ**.

Gemini Notebook

Code: Xây dựng WaveNet với Keras

Ta dùng vòng lặp để tạo các lớp Conv1D giãn nở:

```
model = keras.Sequential()
model.add(keras.layers.InputLayer(input_shape=[None, 1]))

# Cac lop tích chap gian nò (1, 2, 4, 8) lạp lại
for rate in (1, 2, 4, 8) * 2:
    model.add(keras.layers.Conv1D(
        filters=20, kernel_size=2, padding="causal",
        activation="relu", dilation_rate=rate))

model.add(keras.layers.Conv1D(filters=10, kernel_size=1))
model.add(keras.layers.Dense(1)) # Lop du doan
```



Lưu ý quan trọng: Tham số padding='causal' cực kỳ quan trọng ở Time Series. Nó chèn 0 vào bên trái để Kernel không bao giờ với tới thông tin của 'tương lai'.

Gemini Notebook

Tổng kết Kiến thức (Phần 1 - RNN Cơ bản)



Sequence Data

Đòi hỏi mô hình có bộ nhớ để theo dõi thứ tự của dữ liệu.



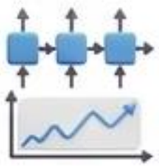
RNN Cơ bản

- Chia sẻ trọng số qua các bước thời gian.
- Dùng Trạng thái ẩn h(t) để mang thông tin tới tương lai.



Ứng dụng RNN đa dạng

- Seq2Seq: Dự báo, Dịch máy
- Seq2Vec: Phân tích cảm xúc
- Encoder-Decoder: Chatbot



BPTT & Tiền xử lý

- BPTT: Mở cuộn mạng qua không gian thời gian (dễ gây Gradient bốc hơi hoặc bùng nổ khi chuỗi quá dài).
- Tiền xử lý (Differencing): Rất hữu ích để loại bỏ tính thời vụ và xu hướng trước khi đưa vào mạng.

Gemini Notebook

Tổng kết Kiến thức (Phần 2 - Các Kiến trúc Tiên tiến)



Trí nhớ Dài Hạn (LSTM & GRU)

Cơ chế: Thiết kế Tế bào thông minh với các "Cổng" (Gate) động đóng mở bằng hàm Sigmoid.

Giá trị cốt lõi: Giúp bảo toàn Gradient đi xa, duy trì thông tin qua nhiều bước thời gian một cách hiệu quả, mang lại "**Trí nhớ Dài Hạn**".



Bứt phá Tốc độ (CNN 1D & WaveNet)

Cơ chế: Một hướng đi song song (không tuần tự), xử lý nhanh hơn RNN rất nhiều.

Giá trị cốt lõi: Sử dụng Tích chập giãn nở (Dilated Convolutions) để nắm bắt và bao phủ chuỗi độ dài hàng chục ngàn bước một cách nhẹ nhàng (Ví dụ: Ứng dụng Âm thanh).

Gemini Notebook